CPSC 4970 – C++ Programming
Homework 6

Your solution to each problem should include the C++ source code and the screenshot of a test run.

1. (NumDays Class) Design a class called `NumDays`. The class's purpose is to store a value that represents a number of work hours and convert it to a number of days. For example, 8 hours would be converted to 1 day, 12 hours would be converted to 1.5 days, and 18 hours would be converted to 2.25 days. The class should have a constructor that accepts a number of hours, as well as member functions for storing and retrieving the hours and days. The class should also have the following overloaded operators:

- + Addition operator. When two `NumDays` objects are added together.
- - Subtraction operator. When one `NumDays` object is subtracted from another.
- ++ Prefix and postfix increment operators. These operators should increment the number of hours stored in the object. When incremented, the number of days should be automatically recalculated.
- -- Prefix and postfix decrement operators. These operators should decrement the number of hours stored in the object. When decremented, the number of days should be automatically recalculated.

2. (Time Off) Design a class named `TimeOff`. The purpose of the class is to track an employee's sick leave and vacation. It should have, as members, the following instances of the `NumDays` class described in Problem 1:

`maxSickDays`	A `NumDays` object that records the maximum number of days of sick leave the employee may take.
`sickTaken`	A `NumDays` object that records the number of days of sick leave the employee has already taken.
`maxVacation`	A `NumDays` object that records the maximum number of days of paid vacation the employee may take.
`vacTaken`	A `NumDays` object that records the number of days of paid vacation the employee has already taken.

Additionally, the class should have members for holding the employee's name and identification number. It should have an appropriate constructor and member functions for storing and retrieving data in any of the member objects.

Input Validation: Company policy states that an employee may not accumulate more than 240 hours of vacation. The class should not allow the `maxVacation` object to store a value greater than this amount.

3. (`FeetInches` Modification) Modify the `FeetInches` class discussed in this chapter so it overloads the following operators:

```
<=

>=

!=
```

Demonstrate the class's capabilities in a simple program.

4. (Corporate Sales) A corporation has six divisions, each responsible for sales to different geographic locations. Design a `DivSales` class that keeps sales data for a division, with the following members:

- An array with four elements for holding four quarters of sales figures for the division.
- A private static variable for holding the total corporate sales for all divisions for the entire year.
- A member function that takes four arguments, each assumed to be the sales for a quarter. The value of the arguments should be copied into the array that holds the sales data. The total of the four arguments should be added to the static variable that holds the total yearly corporate sales.
- A function that takes an integer argument within the range of 0–3. The argument is to be used as a subscript into the division quarterly sales array. The function should return the value of the array element with that subscript.

Write a program that creates an array of six `DivSales` objects. The program should ask the user to enter the sales for four quarters for each division. After the data are entered, the program should display a table showing the division sales for each quarter. The program should then display the total corporate sales for the year.

Input Validation: Only accept positive values for quarterly sales figures.

5. (Pure Abstract Base Class) Define a pure abstract base class called `BasicShape`. The `BasicShape` class should have the following members:

Private Member Variable:

`area`: A `double` used to hold the shape's area.

Public Member Functions:

`getArea`: This function should return the value in the member variable `area`.

`calcArea`: This function should be a pure virtual function.

Next, define a class named `Circle`. It should be derived from the `BasicShape` class. It should have the following members:

Private Member Variables:

`centerX`: a long integer used to hold the x coordinate of the circle's center

`centerY`: a long integer used to hold the y coordinate of the circle's center

`radius`: a double used to hold the circle's radius

Public Member Functions:

constructor: accepts values for `centerX`, `centerY`, and `radius`. Should call the overridden `calcArea` function described below.

`getCenterX`: returns the value in `centerX`

`getCenterY`: returns the value in `centerY`

`calcArea`: calculates the area of the circle (area = 3.14159 `* radius * radius`) and stores the result in the inherited member `area`.

Next, define a class named `Rectangle`. It should be derived from the `BasicShape` class. It should have the following members:

Private Member Variables:

`width`: a long integer used to hold the width of the rectangle

`length`: a long integer used to hold the length of the rectangle

Public Member Functions:

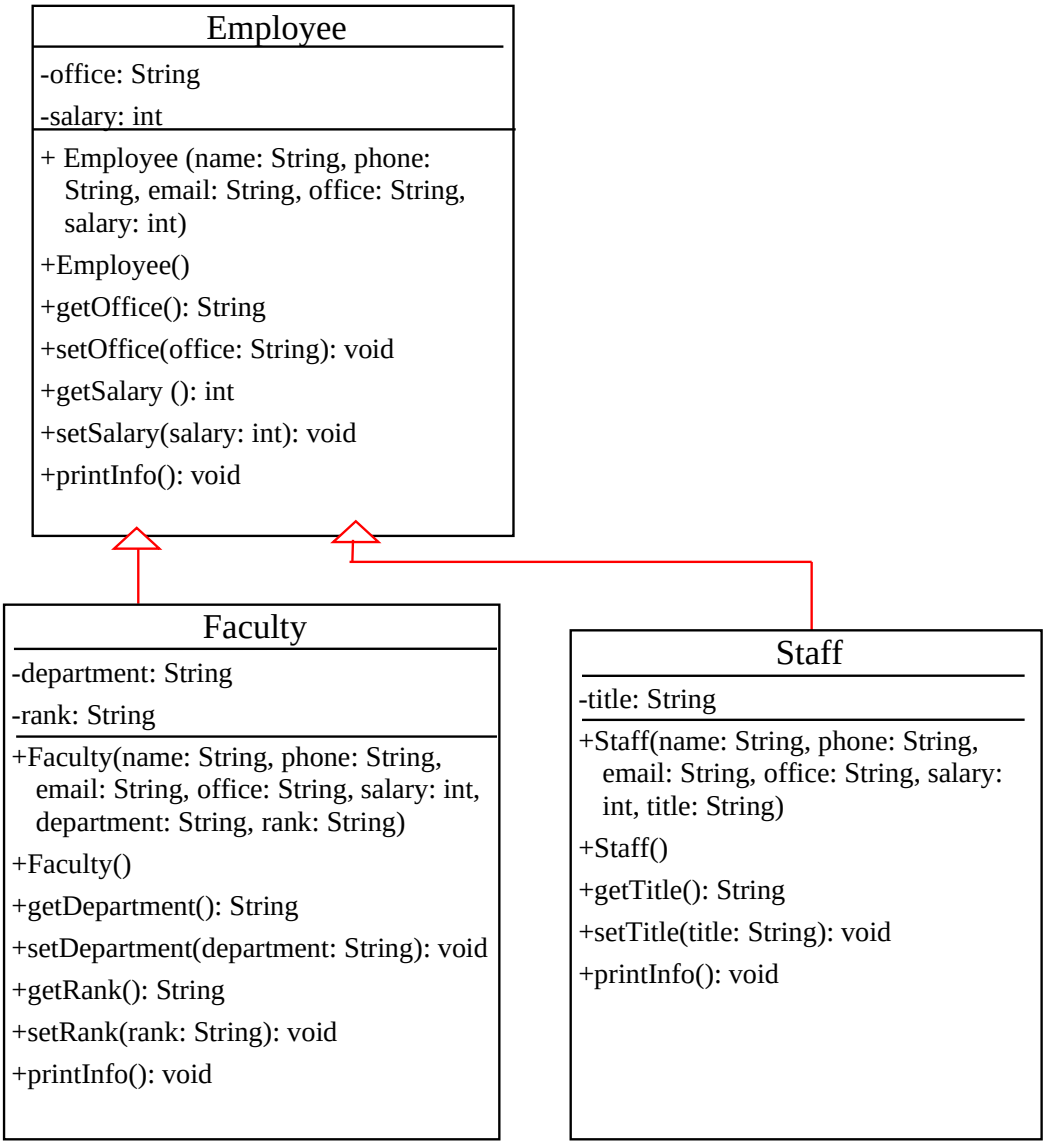
constructor: accepts values for width and length. Should call the overridden `calcArea` function described below.

`getWidth`: returns the value

`calcArea`: calculates the area of the rectangle (area = length * width) and stores the result in the inherited member `area`.

After you have created these classes, create a driver program that defines a `Circle` object and a `Rectangle` object. Demonstrate that each object properly calculates and reports its area.

6. (The Employee Class) Continue with the project we did in class, design a class named **Employee**, which inherits **Person**. Design two child classes for **Employee**, named **Faculty** and **Staff**. The UML diagram for the three classes is as follows.



Write a test program that creates a **Faculty** object and a **Staff** object using one of their constructors and invokes their **printInfo()** methods.

.